

# ISE 555: Advanced Deterministic Systems Optimization — Lecture Notes

September 14 and 16, 2026

## Contents

|           |                                                     |          |
|-----------|-----------------------------------------------------|----------|
| <b>1</b>  | <b>The operations research workflow</b>             | <b>2</b> |
| <b>2</b>  | <b>Why elementary calculus is not always enough</b> | <b>2</b> |
| <b>3</b>  | <b>Optimization problems and their formulation</b>  | <b>2</b> |
| <b>4</b>  | <b>A two-variable linear program</b>                | <b>3</b> |
| <b>5</b>  | <b>Cabinet manufacturing</b>                        | <b>5</b> |
| <b>6</b>  | <b>Insurance workload allocation</b>                | <b>5</b> |
| <b>7</b>  | <b>Formulating different kinds of decisions</b>     | <b>7</b> |
| <b>8</b>  | <b>Variables, constraints and feasibility</b>       | <b>7</b> |
| <b>9</b>  | <b>Optimality: global, local and strict</b>         | <b>7</b> |
| <b>10</b> | <b>When a local solution is useful</b>              | <b>8</b> |

# Optimization, Linear Programming, Workload Allocation and Feasibility

September 14, 2026

## 1 The operations research workflow

The five phases of an operations research study are:

1. Formulate the problem.
2. Construct a mathematical model.
3. Derive a solution for the model through optimization.
4. Test the model and its solution.
5. Implement the solution.

Modeling translates a practical decision into variables, an objective and constraints. Solving the mathematical model is only one phase: its assumptions and the usefulness of its answer still need to be checked.

## 2 Why elementary calculus is not always enough

Consider maximizing  $y = f(x)$  subject to  $0 \leq x \leq x_1$ . A function can have several turning points satisfying  $dy/dx = 0$ . Here  $x_1$  denotes the upper endpoint; it is not the first component of the decision vector used in the later examples.

Practical difficulties include checking endpoints separately, distinguishing local optima from a global optimum, computing higher derivatives to classify stationary points, controlling numerical differentiation errors, and handling discrete variables for which the usual derivative is not defined on the decision domain.

For a differentiable function, an interior local optimum must satisfy  $f'(x) = 0$ , but that equation alone does not guarantee an optimum. Endpoints and nondifferentiable points require separate attention. Even the second derivative test can be inconclusive.

## 3 Optimization problems and their formulation

Optimization includes nonlinear programming, linear programming (the initial focus), stochastic optimization and multi-objective optimization. Stochastic models involve uncertainty; multiple objectives may conflict.

**Definition — Deterministic systems optimization.** A system consists of interacting components. In a deterministic model, its behavior is determined by specified inputs and parameters rather than inherent randomness. Optimization seeks the most effective choice according to an objective.

To formulate and solve a problem: identify decision variables; express the objective; express constraints; classify the problem; apply an appropriate algorithm; and communicate what the solution means in the original setting.

**Our standard problem-formulation method.** Use the same sequence in every worked problem. Keep given parameters separate from quantities we are allowed to choose; writing an equation is meaningful only after its symbols and units have been defined.

1. **Decision and scope.** State what must be decided, over which period and for which products, workers or tasks. If the period is not given, record that omission rather than inventing one.
2. **Given data.** List coefficients, available resources and requirements with their units and source. These are parameters, not decision variables.
3. **Decision variables.** Write one precise sentence per variable: meaning, index, units and domain. A number of units, a fraction and a yes/no decision are different objects.
4. **Objective.** Write what is minimized or maximized, with units. Distinguish revenue from profit, and total effort from elapsed completion time.
5. **Constraints.** Translate each requirement separately and explain its direction: consumption at most capacity gives  $\leq$ ; a minimum requirement gives  $\geq$ ; full allocation or exact balance gives  $=$ . Check that both sides have matching units.
6. **Domain and assumptions.** State nonnegativity, continuity, integrality or binary restrictions explicitly. Explain divisibility, proportional coefficients and any scheduling assumptions needed by the model.
7. **Validate and interpret.** Check all stated requirements appear, test an easy feasible or infeasible candidate, then solve. Substitute the result into every constraint and the objective; communicate the decision and the available optimality guarantee.

Common algorithm categories include optimization algorithms, approximation algorithms and heuristics.

The guarantee depends on the algorithm and problem class. A heuristic can produce a useful feasible answer without proving global optimality; an approximation guarantee must be stated explicitly when one exists.

## 4 A two-variable linear program

$$\begin{array}{ll} \max & f(x) = 2x_1 + x_2 \\ \text{subject to} & x_1 + x_2 \leq 1, \\ & x_1, x_2 \geq 0. \end{array}$$

The feasible region is the triangle with vertices  $A = (1, 0)$ ,  $B = (0, 1)$  and  $C = (0, 0)$ . This region lies in the first quadrant.

| Vertex | Coordinates | Objective value |
|--------|-------------|-----------------|
| A      | (1, 0)      | 2               |
| B      | (0, 1)      | 1               |
| C      | (0, 0)      | 0               |

The optimal solution is  $x_1 = 1$ ,  $x_2 = 0$ , with optimal value 2.

For every feasible point,  $2x_1 + x_2 \leq 2(x_1 + x_2) \leq 2$ . Point A attains this bound, which directly proves optimality without relying on a general vertex theorem.

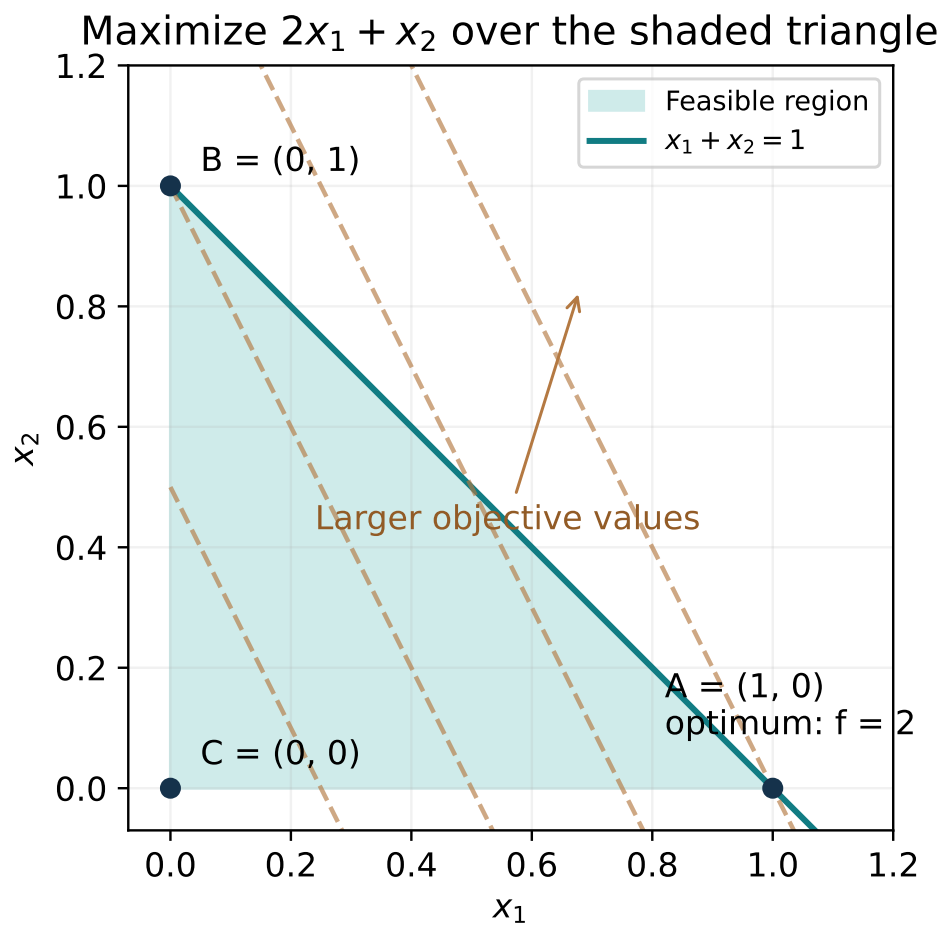


Figure 1: Feasible triangle and objective level lines for maximizing  $2x_1 + x_2$ .

## 5 Cabinet manufacturing

Let  $x_1, x_2, x_3, x_4$  denote the quantities produced of the four products.

| Product              | Variable | Wood requirement | Labor requirement | Revenue |
|----------------------|----------|------------------|-------------------|---------|
| Bookshelf            | $x_1$    | 10               | 2                 | 100     |
| Cabinet with doors   | $x_2$    | 12               | 4                 | 150     |
| Cabinet with drawers | $x_3$    | 25               | 8                 | 200     |
| Custom cabinet       | $x_4$    | 20               | 12                | 400     |

$$\begin{aligned}
 \max \quad & Z = 100x_1 + 150x_2 + 200x_3 + 400x_4 \\
 \text{subject to} \quad & 10x_1 + 12x_2 + 25x_3 + 20x_4 \leq 5000, \\
 & 2x_1 + 4x_2 + 8x_3 + 12x_4 \leq 1500, \\
 & x_1, x_2, x_3, x_4 \geq 0.
 \end{aligned}$$

The optimal solution is  $(375, 0, 0, 62.5)$  and  $Z = \$62,500$ . Wood and labor are measured in their respective resource units. The coefficients and capacities must use consistent units.

**Numerical check.** Wood consumption is  $10(375) + 20(62.5) = 5000$ ; labor consumption is  $2(375) + 12(62.5) = 1500$ . Revenue is  $100(375) + 400(62.5) = 62500$ .

**Optimality certificate.** Multiply the wood constraint by 5 and the labor constraint by 25. Their sum gives  $100x_1 + 160x_2 + 325x_3 + 400x_4 \leq 62500$ . Nonnegativity implies  $Z \leq 62500$ , and this solution attains that bound.

**Divisibility.** This is a continuous LP, so fractional values are permitted. If complete cabinets must be produced, integer restrictions are needed. Rounding a continuous solution does not generally preserve optimality or feasibility; no integer solution is claimed here.

A general linear programming maximization form is

$$\max Z = \sum_{j=1}^n c_j x_j, \quad \sum_{j=1}^n a_{ij} x_j \leq b_i \quad (i = 1, \dots, m), \quad x_j \geq 0 \quad (j = 1, \dots, n).$$

$n$  counts decision variables and  $m$  counts constraints;  $c_j$  is an objective coefficient,  $a_{ij}$  a constraint coefficient and  $b_i$  a right-hand-side limit. Other LP formulations may use minimization or equalities.

September 16, 2026

## 6 Insurance workload allocation

Five workers can handle information requests (P), new policies (Q) and claims (R). The table gives processing-time coefficients, expressed in a common time unit.

| Worker | P  | Q  | R  |
|--------|----|----|----|
| 1      | 10 | 28 | 31 |
| 2      | 15 | 22 | 42 |
| 3      | 13 | 18 | 35 |
| 4      | 19 | 25 | 29 |
| 5      | 17 | 23 | 33 |

Let  $P_i$ ,  $Q_i$  and  $R_i$  be the fractions of the respective workloads assigned to worker  $i$ .

In the diagram,  $a_i$ ,  $b_i$  and  $c_i$  are the P, Q and R time coefficients for worker  $i$ , and  $L_i$  is that worker's total assigned processing time.

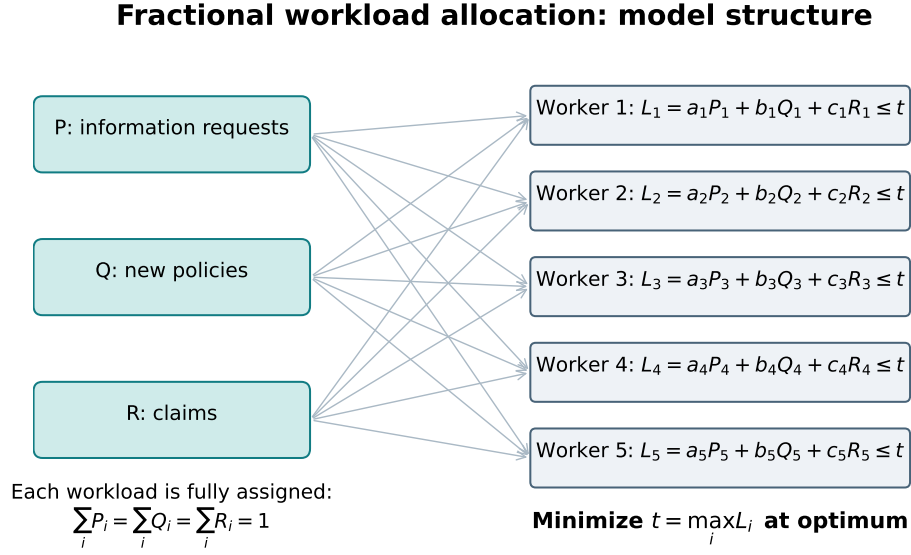


Figure 2: Each task category is split across workers;  $t$  bounds every worker load. Schematic, not a computed allocation.

The objective is  $\min Z = t$ . Worker 1 must satisfy  $10P_1 + 28Q_1 + 31R_1 \leq t$ , with analogous constraints for every worker. Each workload is fully assigned:

$$\sum_{i=1}^5 P_i = 1, \quad \sum_{i=1}^5 Q_i = 1, \quad \sum_{i=1}^5 R_i = 1, \quad P_i, Q_i, R_i \geq 0.$$

**Meaning of fractions and  $t$ .** A fraction of 0.3 means 30% of that task type's total workload. Multiplying a fraction by a time coefficient assumes proportional processing time. The model bounds every worker's load by  $t$ ; minimizing  $t$  minimizes the maximum worker load (makespan under simultaneous work), not the sum of all workers' hours. The formulation assumes that each worker performs their assigned tasks sequentially, while different workers operate simultaneously.

**Expanded model.** Worker 4 has a P coefficient of 19. The remaining worker constraints are

$$\begin{aligned} 15P_2 + 22Q_2 + 42R_2 &\leq t, \\ 13P_3 + 18Q_3 + 35R_3 &\leq t, \\ 19P_4 + 25Q_4 + 29R_4 &\leq t, \\ 17P_5 + 23Q_5 + 33R_5 &\leq t. \end{aligned}$$

The equivalent form  $10P_1 + 28Q_1 + 31R_1 - t \leq 0$  is correct:  $t$  is itself a decision variable. Nonnegative loads already force  $t \geq 0$ .

**Equality as two inequalities.** An exact assignment requirement  $\sum_i P_i = 1$  can be expressed as two inequalities:

$$\sum_i P_i \leq 1, \quad -\sum_i P_i \leq -1.$$

The first inequality prevents over-allocation; the second requires allocation of at least the entire workload. Together they ensure full assignment.

**Validation.** Before solving, check that there are 15 allocation variables plus  $t$ , five worker-capacity constraints, three full-assignment equalities and nonnegative allocations. For any computed answer, recompute all three fraction sums and five loads. Report the largest load as the objective and state that the interpretation assumes divisible workloads and simultaneous workers.

## 7 Formulating different kinds of decisions

**Two formulations, different meanings.** In manufacturing,  $x_j$  is a production quantity: a resource coefficient times  $x_j$  is resource use, and total use must not exceed availability. In insurance,  $P_i$  is a fraction of one task category: fractions across workers sum to 1, not across categories for a single worker. A worker's time combines three task categories and must be at most  $t$ .

**Modeling mistakes to avoid.** Nonnegativity is not integrality. Revenue is not profit unless costs are accounted for. In the insurance model, imposing  $P_i + Q_i + R_i = 1$  for each worker would change the problem: the three fractions refer to different workload totals. Their sum is not a worker's utilization. Utilization is modeled through time,  $a_i P_i + b_i Q_i + c_i R_i \leq t$ .

## 8 Variables, constraints and feasibility

Feasibility describes which decisions satisfy the model. Optimality compares the objective values of feasible decisions.

A decision vector  $x$  contains  $N$  variables;  $x_i$  denotes one component. Real-valued decisions satisfy  $x \in \mathbb{R}^N$ , with  $x_i \in \mathbb{R}$ . Integer-valued decisions satisfy  $x \in \mathbb{Z}^N$ .

**Binary variables.** A binary vector satisfies  $x \in \{0, 1\}^N$ . Equivalently, require  $x \in \mathbb{Z}^N$  and  $0 \leq x_i \leq 1$  for every component. Integer membership alone allows values other than 0 and 1.

Use  $\mathcal{E}$  for equality-constraint indices and  $\mathcal{J}$  for inequality indices. We use the convention

$$g_i(x) = 0 \quad (i \in \mathcal{E}), \quad g_i(x) \geq 0 \quad (i \in \mathcal{J}).$$

**Definition — Feasible point and feasible set.** A feasible point belongs to the variable domain and satisfies every constraint. The feasible set  $S$  is the set of all such points.

The direction  $g_i(x) \geq 0$  is a convention, compatible with  $Ax \leq b$  by writing  $g_i(x) = b_i - (Ax)_i$ . An equality  $h(x) = 0$  can be expressed as  $h(x) \geq 0$  and  $-h(x) \geq 0$ .

## 9 Optimality: global, local and strict

The minimization formulation is  $\min_{x \in S} f(x)$ ; "s.t." abbreviates "subject to".

**Maximization to minimization.** Negate the objective to convert a maximization problem into a minimization problem over the same feasible set:

$$\max_{x \in S} f(x) = -\min_{x \in S} [-f(x)].$$

The optimal decisions coincide after this conversion, while the optimal values have opposite signs. For example, maximizing  $x$  on  $[0,1]$  and minimizing  $-x$  on  $[0,1]$  both select  $x=1$ ; their optimal values are 1 and -1.

**Definition — Global minimizer.** A feasible point  $x^* \in S$  is a global minimizer if  $f(x^*) \leq f(x)$  for every  $x \in S$ .

**Definition — Local minimizer.** A feasible point  $x^* \in S$  is a local minimizer if there exists  $\epsilon > 0$  such that  $f(x^*) \leq f(x)$  for every  $x \in S$  satisfying  $\|x - x^*\| < \epsilon$ .

**Strict minimizers.** A strict global minimizer  $x^* \in S$  satisfies  $f(x^*) < f(x)$  for every  $x \in S$  with  $x \neq x^*$ . A strict local minimizer satisfies this inequality for all feasible  $x$  such that  $0 < \|x - x^*\| < \epsilon$ , for some  $\epsilon > 0$ . The point itself must be excluded from a strict comparison because a value cannot be strictly less than itself.

## 10 When a local solution is useful

A local solution may be sufficient when time is limited, a feasible answer is needed, or the solution space is very large.

This is a practical stopping consideration, not a proof that a local solution is globally optimal. Report feasibility and the available optimality guarantee separately.